# Do Spiking Networks Help Flow-Based Intrusion Detection? A Duplicate-Controlled Comparison on CSE-CIC-IDS2018

Özgür Uçar
ucar45ozgur@gmail.com

***Abstract*—Spiking neural networks (SNNs) are often proposed for network intrusion detection because neuromorphic hardware could run them at low power. Most reported results come from random row splits of benchmarks that contain many duplicate flows, and they rarely include tree ensembles as baselines. We revisit the question on CSE-CIC-IDS2018 (16.2 million flows, 13 classes). Rows with identical feature vectors stay in one partition, fitting, calibration, threshold selection and testing use five disjoint partitions, and every model is judged at the same benign false-alarm rate (FAR). We compare a three-branch ensemble SNN with a parameter-matched multilayer perceptron (MLP), a spike-input control that feeds the same spike encodings to a non-spiking network, and three tree ensembles. At a 5% FAR the SNN reaches a macro-F1 of $0.769 \pm 0.022$ over three seeds, against $0.756$ for the matched MLP, $0.787$ for the spike-input control and $0.788$–$0.808$ for the trees. With three seeds we find no detectable difference between the SNN and either neural baseline. XGBoost scores highest in every seed. Two classes whose feature vectors almost all recur under other labels account for much of the spread between models. Shortening the SNN's time window from 12 to 4 steps leaves macro-F1 unchanged and cuts synaptic events 2.9-fold, yet under published Loihi per-operation costs the SNN still needs about 3 $\mu$J per flow against 0.4 $\mu$J for the MLP. As earlier work found for conventional detectors, the frozen models transfer poorly to CIC-IDS2017, detecting 23%–32% of attack flows after threshold recalibration. Code, records and checkpoints are released.**

***Index Terms*—intrusion detection, spiking neural networks, CSE-CIC-IDS2018, data leakage, cross-dataset evaluation, neuromorphic computing**

## I. Introduction

Network intrusion detection systems (NIDS) that classify flow records are attractive because flow export is cheap and already deployed in most networks. Learning-based detectors trained on public benchmarks such as CSE-CIC-IDS2018 [1] routinely report accuracies above 99%. Spiking neural networks [2] have joined this line of work, motivated by the low power that neuromorphic chips such as Loihi [3], [4] promise for always-on monitoring.

Two problems make many of these numbers hard to interpret. The CIC datasets contain labelling and feature-extraction errors [5], [6], [7], and a large share of their rows are exact duplicates. When a random row split puts copies of the same vector into both training and test sets, the test score partly measures memorisation [8], [9]. SNN papers also tend to compare against convolutional or recurrent networks trained in a similar way, while gradient-boosted trees, which remain hard to beat on tabular data [10], [11], are left out. Reporting accuracy at the default argmax decision hides the operating point that matters to an analyst, namely how many attacks are caught at a fixed rate of false alarms [12].

This paper asks a narrow question. If the evaluation keeps duplicate vectors out of the test set, calibrates every model the same way and fixes the false-alarm rate, does an ensemble SNN offer an accuracy or efficiency advantage on CSE-CIC-IDS2018? Our contributions are:

- A duplicate-controlled protocol for CSE-CIC-IDS2018, with separate partitions for model selection, probability calibration, alarm calibration and test, applied here to SNN-based detection.
- A spike-input control that feeds the SNN's encodings to a non-spiking network. Together with a parameter-matched MLP and three tree ensembles, it separates the contribution of the encoding from that of the membrane dynamics. At three seeds we find no evidence that the dynamics add accuracy.
- An audit of duplicated feature vectors that quantifies how much of the remaining error comes from vectors carrying conflicting labels, extending earlier analyses of labelling errors in this dataset [6].
- A time-window ablation ($T \in \{4, 8, 12\}$) with energy estimates under two cost models, which disagree on whether the SNN is cheaper than the MLP.
- A transfer test of the frozen detectors on CIC-IDS2017 and a label-budget study of local adaptation, which confirm for SNNs the cross-dataset failure reported for other learners [13], [14].

Our results are mixed. The SNN is a competent detector and runs with a third of its time window at no measurable loss, but it does not reach the tree ensembles, and we cannot distinguish it from a non-spiking network fed the same encoded input.

## II. Related Work

**Dataset quality and cross-dataset tests.** Engelen et al. [5] documented flow-construction and labelling errors in CIC-IDS2017, Lanvin et al. [15] showed that such errors change detection results, and Liu et al. [6] extended the analysis to CSE-CIC-IDS2018. Flood et al. [7] catalogue structural design problems in benchmark NIDS datasets. Arp et al. [9] list sampling bias and data snooping among common pitfalls, Sommer and Paxson [16] warned that closed-world benchmark

success rarely carries over to real networks, and Apruzzese et al. [12] argue for evaluations that reflect deployment. Cross-dataset failure is well documented here. D'hooge et al. trained on CSE-CIC-IDS2018 and tested on CIC-IDS2017 and related sets [13], and studied the reverse direction [17]; supervised models rarely learned a representation that carried over. Cantone et al. [14] ran a systematic cross-dataset study over CIC-IDS2017, CSE-CIC-IDS2018 and their corrected versions. Our transfer test repeats this kind of experiment for SNNs.

**Tabular baselines and numerical encodings.** Tree ensembles still match or beat deep networks on most tabular benchmarks [10], [11]. Gorishniy et al. [18] showed that piecewise-linear and periodic embeddings of scalar features substantially improve plain MLPs. Deterministic spike codes of a scalar are, in effect, fixed binning or radial-basis encodings, so a gain from spike encoding may be an instance of this effect.

**Spiking networks and energy.** Surrogate-gradient training [19] made it practical to train SNNs with backpropagation through time, and snnTorch [20] brought this into standard tooling. Patel et al. [21] found that for intrusion detection the choice of spike code matters more than the neuron model. Energy claims for SNNs usually rest on an accumulate costing less than a multiply-accumulate [22]. Dampfhoffer et al. [23] showed with a hardware-aware model that includes memory access that SNNs beat efficient ANN implementations only at high spike sparsity. Loihi figures are given in [3], [4].

**SNN-based intrusion detection.** Zhou and Li [24] compared single-spike temporal-coded networks with tree-ensemble figures taken from earlier work. Wang et al. [25] reported 98.82% accuracy for a convolutional SNN on a six-class subset of CSE-CIC-IDS2018 without a described split, against neural baselines only. Other studies set SNNs against CNNs or small baselines on NSL-KDD and BoT-IoT [26], [27], or report a 1% false-alarm operating point on random splits [28]. Patel et al. [21] benchmarked 27 spiking configurations under capture-aware splits, without non-spiking baselines. Hardware evidence comes from a TrueNorth chip [29] and an FPGA [30]. Our contribution is to bring controls that the NIDS evaluation literature already recommends (duplicate-aware splits, tree baselines, fixed false-alarm operating points and cross-dataset tests) to SNN-based detection, and to add a spike-input control. Where Patel et al. ask which code works best, the control asks whether spiking adds anything beyond the code.

**Calibration and operating points.** We use class-prior correction [31] and temperature scaling [32], and set alarm thresholds in the Neyman–Pearson framing [33]; unlike the umbrella algorithm of Tong et al., our empirical-quantile threshold has no finite-sample guarantee.

## III. Data and Protocol

### A. Dataset and cleaning

We use the ten daily CSV files of CSE-CIC-IDS2018 from the Kaggle mirror solarmainframe/ids-intrusion-csv. After removing repeated header lines and rows with unparseable timestamps, 16,232,929 flows remain. Original labels are mapped to 13 classes; the three web-attack labels (brute force, XSS and SQL injection) are merged into one *Web Attacks* class because together they have only 928 flows. Flow identifiers, IP addresses, source port and timestamp are dropped before learning. Non-finite values are replaced by zero, and we record how often this happens for each column.

TABLE I
Support per class after the feature-group split. Per-class ratios deviate from 60/10/10/20 because whole duplicate groups go to one partition. Only training is capped.

| Class | Train (capped) | Tune | Cal. | Test |
|---|---|---|---|---|
| Benign | 335,515 | 1,362,310 | 1,330,511 | 2,699,900 |
| Bot | 16,651 | 22,981 | 33,237 | 54,821 |
| DDoS-HOIC | 26,260 | 68,256 | 69,218 | 134,836 |
| DDoS-LOIC-UDP | 1,066 | 159 | 181 | 324 |
| DDoS-LOIC-HTTP | 23,516 | 57,929 | 57,319 | 115,366 |
| DoS-GoldenEye | 10,601 | 4,135 | 4,078 | 8,362 |
| DoS-Hulk | 20,762 | 46,577 | 45,539 | 92,601 |
| DoS-SlowHTTPTest | 14,486 | 6,487 | 568 | 11,441 |
| DoS-Slowloris | 6,422 | 1,053 | 1,064 | 2,451 |
| FTP-BruteForce | 16,781 | 1,879 | 6,095 | 7,049 |
| Infiltration | 13,497 | 16,205 | 16,432 | 32,470 |
| SSH-BruteForce | 13,887 | 9,435 | 9,888 | 61,742 |
| Web Attacks | 556 | 87 | 106 | 179 |

### B. Feature-group split

We compute a 128-bit fingerprint of each row's cleaned feature vector and treat all rows with the same fingerprint as one group. Groups, not rows, are assigned to training, tuning, calibration and test partitions in a 60/10/10/20 ratio (split seed 2026). A group is not a network session; it only guarantees that no test vector was seen verbatim during training. It controls exact duplication, not near-duplicates from the same attack episode, hence "duplicate-controlled". Classes with many duplicates shrink a great deal under this rule. DDoS-HOIC, for example, has 686,012 flows but only 198,861 distinct vectors. Table I gives the resulting support.

The training partition is capped at 500,000 rows. Classes with at most 10,000 training rows are kept in full, and the remaining capacity is shared in proportion to class size. No synthetic oversampling is used. The calibration partition is split again by groups, half for fitting probability calibration and half for choosing alarm thresholds, which gives five disjoint partitions in total.

### C. Preprocessing

Every preprocessing step is fitted on the training partition only. Constant columns are removed, columns with absolute skew above 5 get a signed $\log(1 + |x|)$ transform, and a quantile transformer maps each feature to $[0, 1]$. Seventy features remain.

### D. Decision rule

All models output 13 class scores $z$, which we treat as logits (tree ensembles use log-probabilities). Because training was class-capped, we add a prior offset [31], $\tilde{z}_c = z_c +$

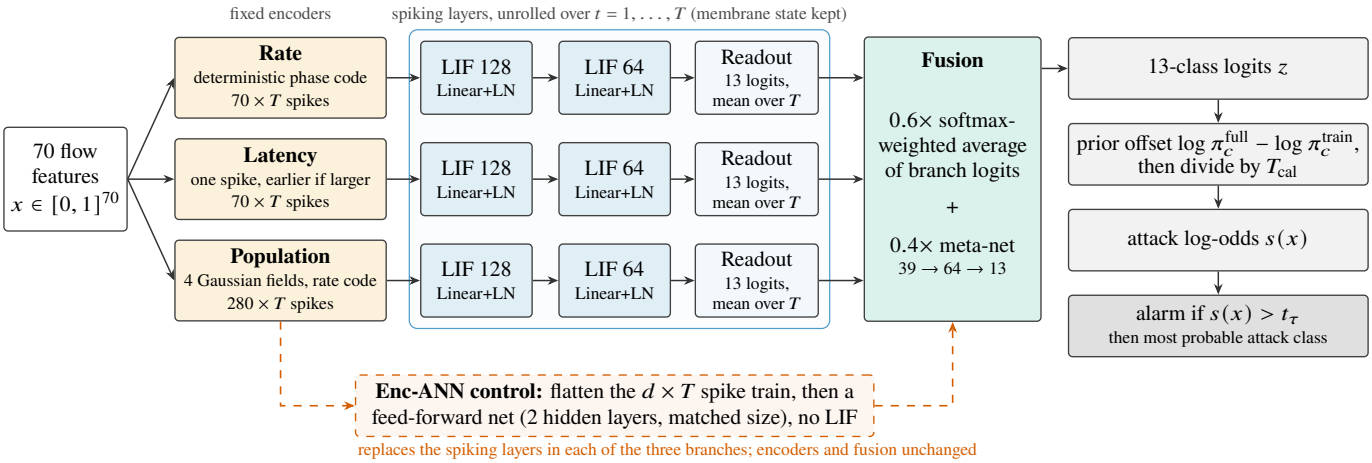



Fig. 1. Ensemble SNN. Three fixed encoders turn the 70 scaled flow features into spike trains over $T$ steps, each branch passes them through two LIF layers and a time-averaged readout, and the branch logits are fused before the prior offset, temperature $T_{\text{cal}}$ and alarm threshold $t_\tau$ are applied. The time window $T$ and the temperature $T_{\text{cal}}$ are unrelated. The dashed path is the spike-input control (Enc-ANN), which replaces the spiking layers with a feed-forward network on the flattened spike train.

$\log \pi_c^{\text{full}} - \log \pi_c^{\text{train}}$. A single temperature $T_{\text{cal}}$ is then fitted by minimising negative log-likelihood on up to 100,000 rows of the probability-calibration half [32]. The alarm score is the attack log-odds

$$s(x) = \log \sum_{c \neq \text{B}} e^{\tilde{z}_c / T_{\text{cal}}} - \tilde{z}_{\text{B}} / T_{\text{cal}}, \tag{1}$$

where B denotes the benign class. We call the share of benign flows that raise an alarm the false-alarm rate (FAR) and the share of attack flows that raise an alarm the detection rate (DR). For a FAR budget $\tau$, the threshold $t_\tau$ is the empirical $(1-\tau)$ quantile of $s$ over benign rows of the alarm-calibration half. A flow raises an alarm if $s(x) > t_\tau$ and is then assigned the class $\arg\max_{c \neq \text{B}} \tilde{z}_c$. We use $\tau \in \{1\%, 5\%, 10\%\}$ and fixed $\tau = 5\%$ as the primary budget before testing. Test labels are never used to pick a threshold, a temperature or a checkpoint.

## IV. Models

**Input.** Every model receives one static vector of 70 flow statistics per flow, as exported by CICFlowMeter. Besides durations, packet and byte counts, inter-arrival statistics and flag counts, the inputs include the destination port and the protocol number as numeric features; only the source port is dropped. Destination port is a known shortcut in testbed captures, since several attacks target a single service. **[Dst Port ablation: E-SNN, Enc-ANN, XGBoost without Dst Port, 3 seeds, MF1/DR at 5% FAR, 2018 test and 2017 transfer.]** No model sees packet sequences; the SNN's time steps only encode each static value as a short spike train.

**Ensemble SNN (E-SNN).** Three branches (Fig. 1) share the same architecture and differ only in how the 70 features become spikes over $T = 12$ time steps. The *rate* branch uses a deterministic phase-accumulator code: feature $x$ adds $x$ to a phase each step and spikes when the phase crosses an integer, with a fixed golden-ratio offset per feature. The *latency* branch emits a single spike at step $\text{round}((1-x)(T-1))$, so larger values spike earlier. The *population* branch expands each feature into four Gaussian receptive fields with centres evenly spaced on $[0,1]$ and width $1/3$, and rate-codes their activations. Each branch has two hidden layers of 128 and 64 leaky integrate-and-fire neurons (decay initialised at 0.9 and learned), layer normalisation before each spiking layer and dropout of 0.2, followed by a linear readout averaged over time. The branch outputs are combined as 0.6 times a softmax-weighted average plus 0.4 times a meta-network ($39 \to 64 \to 13$). This mixture was fixed in an earlier version of the study and not tuned here. The model has 86,013 parameters and is trained with surrogate gradients (fast sigmoid, slope 25) in snnTorch [20].

**Matched MLP.** A perceptron with two hidden layers of 252 units (layer normalisation, GELU, dropout 0.2), a width chosen by parameter count alone to match the E-SNN (85,945 parameters). It sees the quantile-scaled features directly.

**Spike-input control (Enc-ANN).** The same three fixed encoders and fusion head as the E-SNN, but each branch flattens the whole spike train and passes it through a two-hidden-layer feed-forward network whose width matches the parameter count of the corresponding SNN branch (84,503 parameters at $T = 12$). Because the flattened input grows with $T$, the hidden width shrinks (21, 21 and 13 units for the rate, latency and population branches at $T = 12$; 53, 53 and 39 at $T = 4$). Comparing E-SNN with Enc-ANN separates the effect of the encoding from the effect of membrane dynamics.

**Spike-free encoding control (PLE-MLP).** A matched MLP that receives a 16-bin piecewise-linear encoding of each feature [18] instead of spikes, with width again set by parameter count. It tests whether the encoders help because they are spiking or because they bin the input. **[PLE-MLP: MF1, MF1-11, DR at 5% FAR; MF1 at 1%.]**

**Tree ensembles.** Random forest [34] (200 trees, depth

TABLE II
TEST RESULTS AT A 5% FAR BUDGET (MEAN ± S.D., THREE SEEDS). MF1-11 EXCLUDES FTP-BRUTEFORCE AND DOS-SLOWHTTPTEST (SECTION V-B). ACHIEVED TEST FAR IS THE SEED MEAN.

| Model | Macro-F1 | MF1-11 | DR | FAR |
|---|---|---|---|---|
| E-SNN | $0.769 \pm 0.022$ | $0.837 \pm 0.019$ | 0.959 | 0.049 |
| Matched MLP | $0.756 \pm 0.004$ | $0.821 \pm 0.005$ | 0.958 | 0.050 |
| Enc-ANN | $0.787 \pm 0.017$ | $0.848 \pm 0.017$ | 0.963 | 0.049 |
| PLE-MLP | **[TBD]** | **[TBD]** | **[TBD]** | **[TBD]** |
| Random forest | $0.788 \pm 0.001$ | $0.842 \pm 0.001$ | 0.965 | 0.050 |
| CatBoost | $0.791 \pm 0.005$ | $0.843 \pm 0.004$ | 0.965 | 0.049 |
| XGBoost | $0.808 \pm 0.005$ | $0.858 \pm 0.001$ | 0.967 | 0.049 |

TABLE III
PER-CLASS TEST F1 AT A 5% FAR BUDGET, MEAN OF THREE SEEDS. THE EIGHT CLASSES WITH F1 $\geq 0.96$ FOR EVERY MODEL ARE OMITTED (DOS-SLOWLORIS, FOR EXAMPLE, RANGES FROM 0.964 FOR THE E-SNN TO 0.998 FOR XGBOOST).

| Class | E-SNN | MLP | Enc-ANN | RF | CatB. | XGB |
|---|---|---|---|---|---|---|
| DDoS-LOIC-UDP | 0.821 | 0.816 | 0.821 | 0.869 | 0.850 | 0.837 |
| DoS-SlowHTTPTest | 0.552 | 0.519 | 0.625 | 0.747 | 0.757 | 0.734 |
| FTP-BruteForce | 0.243 | 0.271 | 0.274 | 0.234 | 0.255 | 0.337 |
| Infiltration | 0.127 | 0.124 | 0.152 | 0.169 | 0.166 | 0.171 |
| Web Attacks | 0.360 | 0.171 | 0.393 | 0.319 | 0.345 | 0.464 |

25), XGBoost [35] (400 trees, depth 8, learning rate 0.1, row and column subsampling 0.8, histogram method) and CatBoost [36] (600 iterations, depth 8, learning rate 0.05). These are common defaults and were not tuned. The neural models, in contrast, were early-stopped on tuning data and the E-SNN went through a five-variant screen, so the neural side received more tuning effort.

**Training details.** All neural models use AdamW with learning rate $4 \times 10^{-4}$, weight decay $10^{-3}$, batch size 1,024, gradient-norm clipping at 1.0 and plain cross-entropy, for at most 35 epochs. We keep the checkpoint with the best prior-corrected macro-F1 on the tuning partition and stop after 10 epochs without improvement. Every model is trained for three seeds (42, 43, 44); seeds change initialisation and batch order, not the split. **[5 seeds (42–46): E-SNN, Enc-ANN, Matched MLP, $T = 4$ and 12.]** As a further check, the E-SNN and matched MLP were also trained with class-weighted cross-entropy. The full configuration is in `configs/group_ce.json`.

Two follow-up experiments reuse the prepared partitions, preprocessor and decision rule byte for byte; the code refuses to run if the parent's data hashes or package versions differ. The first retrains the E-SNN and Enc-ANN with $T = 4$ and $T = 8$ for all three seeds. The second trains five E-SNN variants at $T = 8$ with seed 42, ranks them by tuning-partition macro-F1 and retrains the winner with all three seeds.

## V. RESULTS

### A. Detection at a fixed FAR budget

Table II reports test results at the primary 5% budget. The achieved test FAR is between 4.9% and 5.0% for every model, so the pooled budget transfers from calibration to test. The Kolmogorov–Smirnov distance between benign calibration and benign test scores is between 0.005 and 0.014 across models and seeds.

All models catch about 96% of attack flows, and DR spans only 0.9 points. More than 99% of the missed attack flows are Infiltration, so DR says little about the models. The separation shows up in macro-F1, which weights rare and difficult classes equally. We pre-declared three comparisons for the E-SNN at $T = 12$. At the 5% budget it differs from the matched MLP by $+1.3$ points (95% CI over seeds $[-3.9, 6.6]$), from Enc-ANN by $-1.8$ ($[-6.3, 2.8]$) and from XGBoost by $-3.9$ ($[-9.0, 1.2]$); none is significant after Holm correction (Welch $p = 0.66$, 0.66, 0.25). Enc-ANN scored higher than the E-SNN on all three seeds, by 1.0 to 2.3 points, a consistent but weak difference (paired $p = 0.05$, Holm 0.15). At the 1% budget the E-SNN is 5.9 points below XGBoost ($[-8.9, -2.8]$, Holm-adjusted $p = 0.035$). With three seeds a Welch test has 80% power only for gaps of about 6 points, so the lack of significance is not evidence of equality. The intervals cover seed variance only. **[Group-bootstrap CIs over test flows (needs per-row test scores from Kaggle).]**

The model order at $\tau = 1\%$ matches Table II (XGBoost 0.848, E-SNN 0.789). At $\tau = 10\%$ Enc-ANN (0.767) moves above random forest (0.763). Macro-F1 rises as the budget tightens because fewer benign flows are pushed into rare attack classes. Without any threshold, the prior-corrected argmax decision runs at a FAR of 0.02% to 0.14% and gives a higher macro-F1 than the 5% budget for every model except random forest (E-SNN 0.788, XGBoost 0.836). The prior offset accounts for the whole calibration effect (E-SNN 0.745 on raw logits); fitted temperatures lie between 0.96 and 1.01. Class-weighted training did not help either neural model ($0.753 \pm 0.004$ for the E-SNN, $0.747 \pm 0.003$ for the MLP).

The pooled FAR hides day-level variation: per capture day it ranges from 0.1% to 24.1% for the E-SNN and from below 0.1% to 32.3% for XGBoost (seed means). Every model exceeds 15% on the two Infiltration days, where unlabelled malicious traffic in the benign class is a plausible cause [6].

### B. Where the errors are

Table III lists per-class F1. Eight of the thirteen classes are detected almost perfectly (F1 $\geq 0.96$) by every model. The differences between models come from the other five: DDoS-LOIC-UDP, DoS-SlowHTTPTest, FTP-BruteForce, Infiltration and Web Attacks.

The feature-group audit explains part of this. All 193,360 FTP-BruteForce flows fall into just 54 distinct vectors, and 193,353 of them lie in groups that also carry another label. For DoS-SlowHTTPTest, 139,884 of 139,890 flows share vectors with other classes, and the three largest groups in the dataset mix these two classes. Both attacks leave short, nearly empty TCP connections that the flow extractor summarises identically, consistent with the labelling problems Liu et al. report [6]. No classifier working on these features can separate such flows, and because whole groups are assigned to one

TABLE IV

TIME WINDOW AND OPERATIONS. TEST MACRO-F1 (MEAN ± S.D., THREE SEEDS) AT 5% AND 1% FAR; OPERATIONS PER FLOW FOR SEED 42, WEIGHTED TO THE TEST CLASS MIX.

| Model | $T$ | MF1 @5% | @1% | ACs | MACs |
|---|---|---|---|---|---|
| E-SNN | 4 | $0.766 \pm 0.015$ | 0.791 | 120,028 | 3,367 |
| E-SNN | 8 | $0.766 \pm 0.014$ | 0.800 | 233,577 | 3,367 |
| E-SNN | 12 | $0.769 \pm 0.022$ | 0.789 | 347,762 | 3,367 |
| Enc-ANN | 4 | $0.787 \pm 0.022$ | 0.815 | 31,349 | 12,352 |
| Enc-ANN | 8 | $0.800 \pm 0.015$ | 0.820 | 31,034 | 6,568 |
| Enc-ANN | 12 | $0.787 \pm 0.017$ | 0.816 | 30,153 | 5,094 |
| Matched MLP | dense | $0.756 \pm 0.004$ | 0.771 | 0 | 84,420 |

partition, the test F1 of these two classes mostly reflects how a few large groups fell in the split. FTP-BruteForce has only six rows in the alarm-calibration half. Removing the two classes (MF1-11 in Table II) narrows the gap between the E-SNN and random forest from 1.9 to 0.5 points and puts Enc-ANN above random forest and CatBoost, so much of the 13-class ranking rests on these two classes. Infiltration is spread over 140,610 groups, and 88,810 of its rows lie in groups that also carry another label.

Across the whole dataset, 68,467 groups carry conflicting labels, and a classifier that predicts the majority label of each group would still be wrong on 1.10% of rows. Because identical inputs cannot be separated even by memorisation, an error below this floor on the same 70 features does not indicate leakage; it implies a different label set or preprocessing, such as de-duplication or class removal, and such results are not directly comparable with ours.

Web Attacks has high ranking quality (one-vs-rest ROC AUC 0.994 for the seed-42 E-SNN, 1.000 for XGBoost) but low F1, because it has only 179 test flows against 2.7 million benign ones. At the 5% budget the E-SNN finds 98 of the 179 attacks but also assigns 661 other flows to the class (precision 0.13); XGBoost finds 160 with 342 false assignments (precision 0.32). Average precision (0.49 and 0.85) reflects usability better than AUC here.

### C. Operation counts and energy

We counted operations with forward hooks on flows from four capture days and weighted the per-class counts by test-partition support. The eight classes measured cover 92% of test rows; LOIC-HTTP, Hulk, SlowHTTPTest, Infiltration and Web Attacks come from other days and are not measured. An earlier 4,400-flow sample (36% benign) gives counts within 2.3%. In a linear layer that receives binary input, each nonzero input contributes one accumulate (AC) per output neuron; every other linear layer contributes multiply-accumulates (MACs). Table IV gives counts per flow for the seed-42 checkpoints.

At $T = 12$ the E-SNN's hidden neurons are silent 72% of the time, yet with 12 steps and three branches it still performs four times as many synaptic events as the MLP performs MACs. Table V prices these counts under four cost models. With 45 nm costs (0.9 pJ per AC, 4.6 pJ per MAC [22]) the E-SNN's linear layers alone cost 0.33 $\mu$J at $T = 12$, below

TABLE V

ESTIMATED ENERGY PER FLOW ($\mu$J). 45 NM: FP32 ACS AND MACS PLUS LAYER NORMALISATION (4 MACS PER ELEMENT) AND, FOR THE E-SNN, ONE MAC PER MEMBRANE UPDATE [22]. +SRAM: ONE 32 KB SRAM READ PER WEIGHT USE. INT8: 8-BIT ARITHMETIC. LOIHI: PUBLISHED PER-OPERATION COSTS [3], WITH LAYER NORMALISATION AND FUSION ON A 45 NM HOST. ESTIMATES, NOT MEASUREMENTS.

| Model | 45 nm | +SRAM | INT8 | Loihi |
|---|---|---|---|---|
| E-SNN, $T = 4$ | 0.18 | 1.41 | 0.007 | 3.1 |
| E-SNN, $T = 8$ | 0.33 | 2.70 | 0.013 | 6.0 |
| E-SNN, $T = 12$ | 0.49 | 4.00 | 0.019 | 9.0 |
| Enc-ANN, $T = 4$ | 0.09 | 0.53 | 0.004 | – |
| Enc-ANN, $T = 8$ | 0.06 | 0.44 | 0.003 | – |
| Enc-ANN, $T = 12$ | 0.05 | 0.41 | 0.002 | – |
| Matched MLP | 0.40 | 1.24 | 0.020 | – |

the MLP's 0.39 $\mu$J. Its layer normalisation and membrane updates run at every time step, though, and with them the E-SNN costs 0.49 $\mu$J, about 23% above the MLP; at $T = 4$ it stays 55% below. Enc-ANN is far cheaper than both because it processes each binary input once instead of propagating it through membrane state. On a single CPU thread, where sparsity brings no saving, the E-SNN took 57 $\mu$s per flow against 1.4 $\mu$s for the MLP in one measurement and 105 against 2.1 $\mu$s in a repeat on the same checkpoints, so it is 40 to 50 times slower.

For neuromorphic hardware we charge each synaptic event (a presynaptic spike times its fan-out) 23.6 pJ and each neuron update 81 pJ, the minimum figures reported for Loihi [3]. The E-SNN then needs about 3.1 to 9.0 $\mu$J per flow, against 0.40 $\mu$J for the MLP on a 45 nm FP32 accelerator, or 1.24 $\mu$J if every weight is also read from SRAM, a cost that Loihi's per-event figure already includes. Since Loihi weights have at most 9 bits, the INT8 MLP (0.020 $\mu$J) is the like-for-like arithmetic point. Enc-ANN stays below the MLP under every model. Synaptic events make up over 90% of the E-SNN total, and about three quarters come from the input encoding, whose spikes would be injected from a host at a cost we do not model. Running the trained E-SNN on chip would also require folding its per-step layer normalisation (under 2% of the estimate); the host round trip, static power and routing are not costed. The 45 nm accumulate is about 26 times cheaper than a Loihi synaptic operation, and which one is used decides the sign of the comparison.

### D. Shorter time windows

Table IV and Fig. 2 repeat the neural comparison with 4 and 8 time steps on the same partitions. For the E-SNN, macro-F1 at the 5% budget is $0.766 \pm 0.015$ at $T = 4$, $0.766 \pm 0.014$ at $T = 8$ and $0.769 \pm 0.022$ at $T = 12$, and at three seeds we detect no difference between them. At the 1% budget the shorter windows score 0.791 and 0.800 against 0.789. Hidden-layer sparsity stays between 72% and 74%, so synaptic events fall almost in proportion to $T$, from 347,762 to 120,028 per flow, a factor of 2.9. The 90% CI of the $T = 4$ minus $T = 12$ difference at 5% is $[-3.7, 3.1]$ points, so a 2-point effect is not excluded. Enc-ANN has the highest mean of the neural

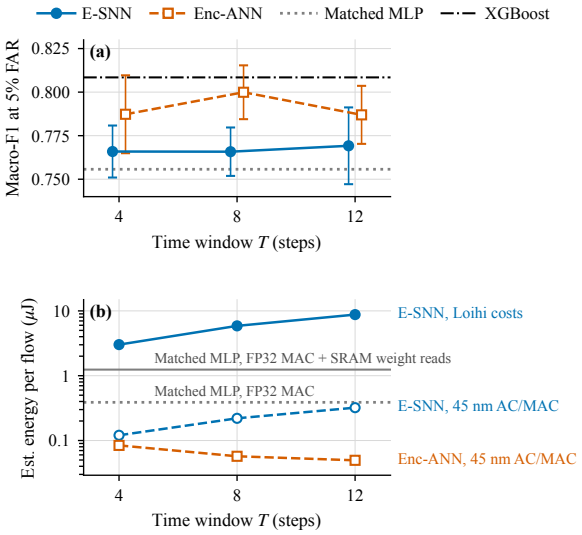



Fig. 2. Effect of the time window $T$. (a) Test macro-F1 at a 5% FAR budget (mean and s.d. over three seeds), with the matched MLP and XGBoost as references. (b) Estimated energy per flow on a log scale: 45 nm costs for the linear layers only, and published Loihi per-operation costs for the E-SNN, on the earlier 4,400-flow sample (within 2.3% of Table V).

models at every window, reaching $0.800 \pm 0.015$ at $T = 8$, though its seed ranges overlap those of the E-SNN.

### *E. Architecture screen*

Five E-SNN variants ($T = 8$, seed 42) were ranked on tuning macro-F1, a rule fixed before the run (tuning / test macro-F1 at 5% FAR): wider layers (256, 128), 0.833 / 0.762; weighted-average fusion only, 0.802 / 0.760; learning rate $10^{-3}$ and dropout 0.1, 0.792 / 0.754; eight receptive fields, 0.787 / 0.768; population branch only, 0.775 / 0.753. The original reached 0.756 on test with seed 42. Every variant fell within the seed spread of the original. A possible cause, which we did not test, is that tuning and test hold different mixes of the conflicting FTP-BruteForce and DoS-SlowHTTPTest groups (Table I). Retrained with three seeds, the wide variant reaches $0.770 \pm 0.014$ at 5% and $0.803 \pm 0.007$ at 1%, against $0.766 \pm 0.014$ and $0.800 \pm 0.001$ for the original at $T = 8$, with 2.5 times as many parameters (217,725). Its seed-42 rerun reproduced the screening result to four decimals (0.7620).

### *F. Transfer to CIC-IDS2017*

We ran the frozen seed-42 models, unchanged, on the CIC-IDS2017 MachineLearningCVE files from the Kaggle mirror sweety18/cicids2017-full-dataset (2,214,468 labelled flows). That dataset was recorded a year earlier on a different testbed with an older version of the same flow extractor. Our copy lacks two of the original files (Tuesday, with the FTP and SSH brute-force attacks, and Thursday morning, with the web attacks), so those classes are absent; the complete or corrected releases [5], [15] would allow a fuller test. The protocol field is missing and is imputed from the destination port. Attack labels are mapped to the 2018 families; PortScan and Heartbleed have no 2018 counterpart. Because the extractor version and

TABLE VI
FROZEN 2018 MODELS ON CIC-IDS2017. "RECAL.": THRESHOLD REFITTED ON HALF OF THE 2017 BENIGN FLOWS (5% BUDGET), ATTACK LABELS UNUSED.

| Model | 2018 threshold FAR | 2018 threshold DR | Recal. FAR | Recal. DR |
|---|---|---|---|---|
| E-SNN ($T = 12$) | 0.262 | 0.630 | 0.050 | 0.230 |
| E-SNN ($T = 4$) | 0.228 | 0.420 | 0.050 | 0.239 |
| XGBoost | 0.252 | 0.494 | 0.050 | 0.317 |

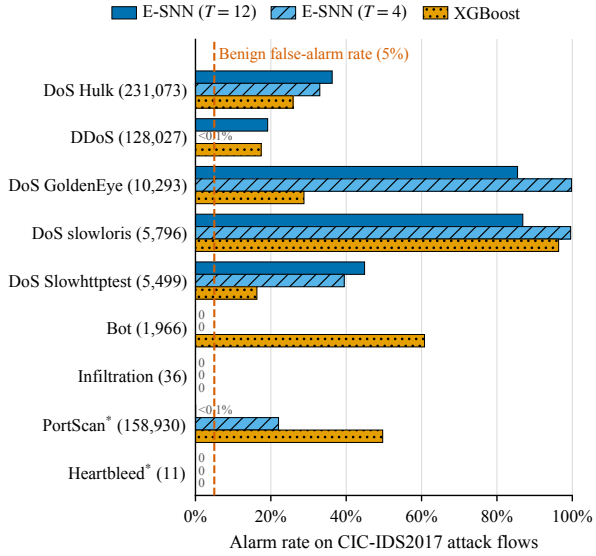



Fig. 3. Share of CIC-IDS2017 attack flows that raise an alarm after the threshold of each frozen 2018 model is refitted to a 5% benign FAR (dashed line). Flow counts are in parentheses; an asterisk marks attack types with no CSE-CIC-IDS2018 counterpart.

the destination ports both differ between the testbeds, the shift we measure mixes network, feature-semantics and port effects. **[Per-feature KS shift, 2017 vs. 2018 benign.]**

Table VI confirms for SNNs the transfer failure reported for conventional learners on this dataset pair [13], [17], [14]. With the 2018 threshold, the FAR rises from the calibrated 5% to between 23% and 26% for all three models. We then gave each model the one adjustment an operator could make without attack labels: refitting the alarm threshold on half of the 2017 benign flows to a 5% budget and evaluating on the other half and on all attacks. DR then falls to 23%–32%. For the SNNs only the slow application-layer DoS attacks transfer well. The 12-step E-SNN still flags 86% of GoldenEye and 87% of Slowloris flows, and the 4-step E-SNN 99.8% and 99.6%, while XGBoost flags only 28.8% of GoldenEye. Volumetric DDoS, DoS-Hulk and Bot traffic is largely missed by the SNNs, whereas XGBoost keeps 61% of Bot flows. No model detects the 36 Infiltration or 11 Heartbleed flows. Fig. 3 breaks the recalibrated DR down by attack type.

To estimate how many local labels an operator would need, we split the CIC-IDS2017 flows by feature-vector fingerprint into an adaptation pool, a calibration partition and a test partition (50/10/40) and drew class-stratified labelled samples

TABLE VII
LOCAL ADAPTATION ON CIC-IDS2017 (TEST PARTITION, MEAN OVER THREE SEEDS) BY LABEL BUDGET, THE LABELLED SHARE OF THE ADAPTATION POOL. BUDGET 0 IS THE FROZEN 2018 MODEL. ALL ROWS USE A 5% FAR THRESHOLD REFITTED ON 2017 CALIBRATION DATA.

| Method | | 0 | 0.1% | 1% | 5% |
|---|---|---|---|---|---|
| E-SNN $T$=4, fine-tuned | FAR | 0.062 | 0.052 | 0.059 | 0.053 |
| | DR | 0.268 | 0.985 | 0.994 | 0.996 |
| | MF1 | 0.188 | 0.564 | 0.592 | 0.707 |
| XGBoost, from scratch | FAR | 0.050 | 0.050 | 0.050 | 0.051 |
| | DR | 0.314 | 0.996 | 1.000 | 1.000 |
| | MF1 | 0.291 | 0.526 | 0.665 | 0.686 |
| XGBoost, continued | FAR | – | 0.051 | 0.050 | 0.057 |
| | DR | – | 0.723 | 0.724 | 0.749 |
| | MF1 | – | 0.510 | 0.494 | 0.483 |

of 0.1%, 1% and 5% of the pool (1,101, 11,026 and 55,153 flows). We call this the label budget, to distinguish it from the FAR budget. PortScan was added as a new class. The temperature and the 5% alarm threshold were refitted on the 2017 calibration partition, and the temperature fit uses up to 100,000 labelled calibration flows that the label budget does not count. **[Rerun with temperature fitted on the label budget only; DR at matched FAR.]** Table VII gives means over three seeds. The frozen E-SNN here has a DR of 0.268 rather than the 0.239 of Table VI, because its threshold comes from the 2017 calibration partition and it is scored on the 40% test partition.

With 1,101 labelled flows, fine-tuning the 4-step E-SNN from its 2018 checkpoint raises DR from 26.8% to 98.5% at a 5.2% FAR, and XGBoost trained from scratch on the same flows reaches 99.6% at 5.0% without any pre-training. The fine-tuned E-SNN matches XGBoost on DoS, DDoS and PortScan traffic but not on Bot flows, which it misses with 0.1% and 1% of the pool and detects at 32% with 5%; XGBoost detects 97% to 99.5% of Bot flows with 1% or more. Continuing the 2018 XGBoost model cannot learn PortScan and stays at 72% to 75% DR. Macro-F1 stays between 0.49 and 0.71 because a 5% FAR on 671,643 benign test flows produces about 33,000 false alarms, which swamp the precision of the small classes. Pool, calibration and test come from the same capture days and attack episodes, so this is a same-capture re-split and an optimistic bound for deployment. The result mainly shows that this dataset becomes easy once some in-domain labels exist; it is not specific to SNN fine-tuning.

## VI. DISCUSSION AND LIMITATIONS

**What the spikes buy.** Within this protocol we find no evidence that membrane dynamics add accuracy over the same encoding fed to a non-spiking network, and the E-SNN's mean gain over the matched MLP lies within seed noise. Enc-ANN has the higher mean and is cheaper under every cost model we tried. This reading is provisional: three seeds give little power, and if the spike-free PLE-MLP matches Enc-ANN, the benefit belongs to input binning rather than to anything specific to spikes [18]. It does not rule out SNN advantages with on-chip encoding or on per-packet sequences, where time steps would be real time; that is the natural next experiment. SNN papers on flow features should include a spike-input control before crediting membrane dynamics.

**Energy needs a hardware model.** A 45 nm adder cost favours the SNN at short windows; the published Loihi cost of a synaptic event, which includes the memory access, favours the MLP by a factor of 2.5 to 22, mainly because the input encoding fans out to 128 neurons. This is an intrusion-detection instance of the sparsity threshold reported by Dampfhoffer et al. [23]. A much sparser input code or cheaper synaptic events would be needed before an energy advantage is plausible, and energy claims should state their cost model.

**Trees remain strong.** Untuned XGBoost had the highest mean at every budget, in line with evidence on tabular data [10], [11], and a neuromorphic detector should be judged against it.

**Limitations.** The protocol in this paper was designed after we had seen test results from an earlier, row-split version of the study, and the time-window and architecture experiments were run after the main test results were known. These are development results on one repeatedly used test partition, so we report every configuration we ran; a fresh held-out dataset is still needed. Feature groups prevent verbatim duplication but not near-duplicates or episode-level leakage within an attack run, and the dataset's timestamps are unverified, so we did not attempt a time-blocked split. Destination port is an input and may act as a testbed-specific shortcut. We used three seeds, a single split seed and fixed tree hyperparameters, and screened only five SNN variants. The CIC-IDS2017 copy is incomplete and was produced by an older extractor. All energy numbers are estimates from operation counts; we made no hardware measurements.

## VII. CONCLUSION

On CSE-CIC-IDS2018 under a duplicate-controlled protocol with matched false-alarm rates, an ensemble SNN detects 96% of attack flows at a 5% FAR. With three seeds we cannot separate it from a parameter-matched MLP or from a non-spiking network fed the same spike encodings, and XGBoost scores higher in every seed. The SNN keeps its accuracy with 4 time steps instead of 12, which cuts its synaptic events by a factor of 2.9, but with published Loihi figures it still costs more energy per flow than the MLP. Much of the remaining error comes from identical vectors under different labels, and, as reported for other learners, no model carries its accuracy to CIC-IDS2017 without local labels.

## REPRODUCIBILITY

Code, configuration, per-run records and the seed-42 checkpoints of the E-SNN ($T$=4 and 12), Enc-ANN, matched MLP and XGBoost are available at https://github.com/ozgurucar/ids2018-ensemble-snn. Checkpoints for the other seeds were not released. Training ran on a single Kaggle GPU; exact package versions are

in `results/v3/requirements-run.lock.txt`. A Docker-packaged service serves the frozen checkpoints with the same 5% threshold. Per-budget, per-day and per-class results beyond those printed are in the per-run records.

## REFERENCES


[1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.

[2] W. Maass, "Networks of spiking neurons: The third generation of neural network models," *Neural Networks*, vol. 10, no. 9, pp. 1659–1671, 1997.

[3] M. Davies *et al.*, "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.

[4] M. Davies, A. Wild, G. Orchard, Y. Sandamirskaya, G. A. Fonseca Guerra, P. Joshi, P. Plank, and S. R. Risbud, "Advancing neuromorphic computing with Loihi: A survey of results and outlook," *Proc. IEEE*, vol. 109, no. 5, pp. 911–934, 2021.

[5] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: the CICIDS2017 case study," in *IEEE Security and Privacy Workshops (SPW)*, 2021, pp. 7–12.

[6] L. Liu, G. Engelen, T. Lynar, D. Essam, and W. Joosen, "Error prevalence in NIDS datasets: A case study on CIC-IDS-2017 and CSE-CIC-IDS-2018," in *IEEE Conf. Communications and Network Security (CNS)*, 2022, pp. 254–262.

[7] R. Flood, G. Engelen, D. Aspinall, and L. Desmet, "Bad design smells in benchmark NIDS datasets," in *IEEE 9th European Symposium on Security and Privacy (EuroS&P)*, 2024, pp. 658–675.

[8] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowledge Discovery from Data*, vol. 6, no. 4, pp. 15:1–15:21, 2012.

[9] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *31st USENIX Security Symposium*, 2022, pp. 3971–3988.

[10] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2022.

[11] R. Shwartz-Ziv and A. Armon, "Tabular data: Deep learning is not all you need," *Information Fusion*, vol. 81, pp. 84–90, 2022.

[12] G. Apruzzese, P. Laskov, and J. Schneider, "SoK: Pragmatic assessment of machine learning for network intrusion detection," in *IEEE 8th European Symposium on Security and Privacy (EuroS&P)*, 2023, pp. 592–614.

[13] L. D'hooge, M. Verkerken, T. Wauters, F. De Turck, and B. Volckaert, "Investigating generalized performance of data-constrained supervised machine learning models on novel, related samples in intrusion detection," *Sensors*, vol. 23, no. 4, p. 1846, 2023.

[14] M. Cantone, C. Marrocco, and A. Bria, "Machine learning in network intrusion detection: A cross-dataset generalization study," *IEEE Access*, vol. 12, pp. 144 489–144 508, 2024.

[15] M. Lanvin, P.-F. Gimenez, Y. Han, F. Majorczyk, L. Mé, and É. Totel, "Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes," in *Risks and Security of Internet and Systems (CRiSIS 2022)*, ser. LNCS, vol. 13857. Springer, 2023, pp. 18–33.

[16] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *IEEE Symposium on Security and Privacy*, 2010, pp. 305–316.

[17] L. D'hooge, M. Verkerken, T. Wauters, F. De Turck, and B. Volckaert, "Characterizing the impact of data-damaged models on generalization strength in intrusion detection," *J. Cybersecurity and Privacy*, vol. 3, no. 2, pp. 118–144, 2023.

[18] Y. Gorishniy, I. Rubachev, and A. Babenko, "On embeddings for numerical features in tabular deep learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022, arXiv:2203.05556.

[19] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks," *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 51–63, 2019.

[20] J. K. Eshraghian, M. Ward, E. O. Neftci, X. Wang, G. Lenz, G. Dwivedi, M. Bennamoun, D. S. Jeong, and W. D. Lu, "Training spiking neural networks using lessons from deep learning," *Proceedings of the IEEE*, vol. 111, no. 9, pp. 1016–1054, 2023.

[21] R. Patel, S. Mitra, D. Amebley, T. Akinrele, S. Dibbo, S. Mittal, and S. Rahimi, "The value of spike timing: A leakage-resistant benchmark of SNN design choices for network intrusion detection," in *IEEE Int. Conf. Cognitive Machine Intelligence (CogMI)*, 2026, to appear. arXiv:2606.01442.

[22] M. Horowitz, "Computing's energy problem (and what we can do about it)," in *IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2014, pp. 10–14.

[23] M. Dampfhoffer, T. Mesquida, A. Valentian, and L. Anghel, "Are SNNs really more energy-efficient than ANNs? An in-depth hardware-aware study," *IEEE Trans. Emerging Topics in Computational Intelligence*, vol. 7, no. 3, pp. 731–741, 2023.

[24] S. Zhou and X. Li, "Spiking neural networks with single-spike temporal-coded neurons for network intrusion detection," in *Proc. 25th Int. Conf. Pattern Recognition (ICPR)*, 2021, pp. 8148–8155, arXiv:2010.07803.

[25] Z. Wang, F. A. Ghaleb, A. Zainal, M. M. Siraj, and X. Lu, "An efficient intrusion detection model based on convolutional spiking neural network," *Scientific Reports*, vol. 14, p. 7054, 2024.

[26] M. Živadinović and D. Simić, "Resource efficient Internet-of-Things intrusion detection with spiking neural networks," in *Position Papers of the 19th Conf. Computer Science and Intelligence Systems (FedCSIS)*, ser. Annals of Computer Science and Information Systems, vol. 40, 2024, pp. 73–78.

[27] A. Fesu, N. Shone, Á. MacDermott, B. Zhou, and M. H. Eiza, "Comparative analysis of SNN and CNN models for energy efficient intrusion detection," in *Proc. 12th IFIP Int. Conf. New Technologies, Mobility and Security (NTMS)*, 2025, pp. 1–7.

[28] M. G. Karthik, V. Keerthika, S. V. Mantena, D. Siri, L. P. Yeluri, K. K. Lella, and B. R. Ganesh, "Energy-efficient intrusion detection with a protocol-aware transformer–spiking hybrid model," *Scientific Reports*, vol. 16, p. 7095, 2026.

[29] M. Z. Alom and T. M. Taha, "Network intrusion detection for cyber security on neuromorphic computing system," in *Proc. Int. Joint Conf. Neural Networks (IJCNN)*, 2017, pp. 3830–3837.

[30] A. Mehrabi and A. van Schaik, "Intrusion detection on industrial networks using neuromorphic computing," in *Proc. Int. Conf. Electrical, Control and Instrumentation Engineering (ICECIE)*, 2024.

[31] M. Saerens, P. Latinne, and C. Decaestecker, "Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure," *Neural Computation*, vol. 14, no. 1, pp. 21–41, 2002.

[32] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. 34th Int. Conf. Machine Learning (ICML)*, 2017, pp. 1321–1330.

[33] X. Tong, Y. Feng, and J. J. Li, "Neyman–Pearson classification algorithms and NP receiver operating characteristics," *Science Advances*, vol. 4, no. 2, p. eaao1659, 2018.

[34] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[35] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[36] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems*, 2018.